

Game Design Document

Soggy Moggy

Verfasser: Julian Gomez **Schule:** SRH Fachschulen, Fachrichtung Game & Multimedia Design (GME-24.01) **Rahmenthema:** 3 — Casual Webgame „Plush Toy Combat“ **Abgabedatum:** 27.04.2026 **Dokumentversion:** 1.0

Inhaltsverzeichnis

1. [Grobkonzept](#1-grobkonzept)

- 1.1 Projektübersicht
- 1.2 Spielidee & Kernaussage
- 1.3 Zielgruppe
- 1.4 Einzigartiges Merkmal (USP)
- 1.5 Plattform & Technologie
- 1.6 Umfang & Levelstruktur

2. [Feinkonzept](#2-feinkonzept)

- 2.1 Spielfluss & Zustände
- 2.2 Steuerung
- 2.3 Spielerfigur
- 2.4 Sprung & Bewegungsmechanik
- 2.5 Aktions-Taste (Z / Rechtsklick)
- 2.6 Plattformen
- 2.7 Gefahren-Mechanik (Hazard)
- 2.8 Leben & Scheitern
- 2.9 Punkte & Fortschritt
- 2.10 Kamera & Weltkoordinaten
- 2.11 Gegner & Gefahren pro Level
- 2.12 Levelstruktur im Detail

3. [Designkonzept](#3-designkonzept)

- 3.1 Visueller Stil & Stimmung
- 3.2 Farbpalette
- 3.3 Sprite-Design: Spielerfigur
- 3.4 Hintergrund & Parallax
- 3.5 Plattform-Design
- 3.6 UI & HUD
- 3.7 Animationssystem
- 3.8 Audio-Konzept
- 3.9 Typografie

- 3.10 Produktionswerkzeuge (Pixel Art)
-

1. Grobkonzept

1.1 Projektübersicht

Feld	Wert
Projekttitel (intern)	Soggy Moggy
Spieltitel (im Spiel)	Soggy Moggy
Genre	Casual Vertical Platformer
Plattform	Web-Browser (HTML5)
Auflösung	480 × 640 px (Hochformat)
Spielsprache	Englisch (UI, Texte, HUD). Migration am 21.04.2026 von Spanisch auf Englisch. Schul-Doku bleibt deutsch.
Engine	Kein Framework — Vanilla JS + HTML Canvas 2D
Ziel-Altersfreigabe	USK 0

1.2 Spielidee & Kernaussage

Soggy Moggy ist ein vertikaler Platformer, in dem der Spieler eine ausgestopfte Katze steuert, die vor einer stetig steigenden Flut nach oben flieht. Im Gegensatz zu klassischen Endlos-Springern (Doodle Jump, etc.) springt die Figur **nicht automatisch**, sondern nur auf aktive Eingabe des Spielers. Das schafft eine direkte, greifbare Kontrolle und macht jede Plattform zu einer bewussten Entscheidung.

Die Atmosphäre vermittelt „cozy danger“: Die Spielwelt ist warm und verspielt gestaltet, die Bedrohung durch die Flut baut aber kontinuierlich Spannung auf. Das Spiel ist in drei thematisch unterschiedliche Level gegliedert, die jeweils eigene Plattfortmtypen und Gefahren einführen.

1.3 Zielgruppe

- **Primär:** Casual-Gamer, 12–30 Jahre, Erfahrung mit Plattformern
- **Sekundär:** Kurzzeit-Spieler (Mobile-Mindset), die schnelle, klare Spielsessions bevorzugen
- **Motivation:** Überleben, Highscore verbessern, alle Level abschließen

1.4 Einzigartiges Merkmal (USP)

Merkmal	Beschreibung
Manueller Sprung	Keine Automatik — volle Kontrolle über Timing
Figur als Kuscheltier	Plüsch-Ästhetik als Kontrast zur Bedrohung
Englischsprachige UI	Klare, kurze Spielsprache; Migration 2026-04-21 von Spanisch
Levelbasiert (nicht Endlos)	3 eigenständige Level mit je eigener Gefahr
Schwierigkeitsgrade	Drei Modi (Explorer, Adventurer, Enlightened) mit globalen Multiplikatoren auf Hazard-Speed, Wespen-Anzahl und Cloud-Drift

Action-Taste (Z)	Ballon fangen, Wespen abwehren, Outro-Trigger am Levelende aktivieren
------------------	---

1.5 Plattform & Technologie

Das Spiel läuft vollständig im Browser ohne Plug-ins oder Build-Tools.

- **Sprache:** JavaScript (ES2022+), HTML5, CSS (nur Layout)
- **Rendering:** HTML Canvas 2D API (480×640 px, gesetzt via JS-Attribute)
- **Audio:** HTMLAudio API (file:// kompatibel, kein XHR, läuft auf Firefox und Chromium)
- **Physik:** Eigenimplementierung (semi-fixes Zeitschritt-Modell, Delta-Cap 50 ms)
- **Hosting:** GitHub Pages
- **Keine externen Bibliotheken**

1.6 Umfang & Levelstruktur

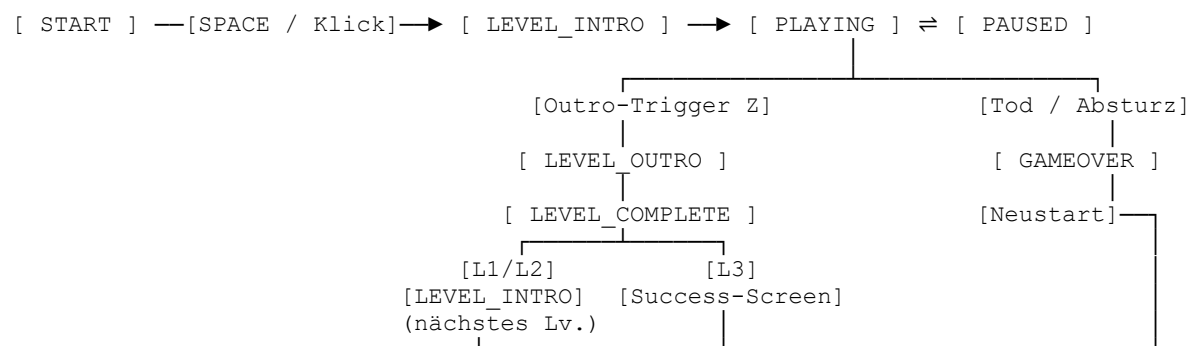
Das Spiel enthält drei vollständige Spiellevel, je mit eigenem visuellen Thema und eigener Kernmechanik:

Level	Titel	Setting	Besondere Gefahr
1	City	Stadtgebäude, Jalousien	Steigender Smog
2	Elevator Shaft	Aufzugschacht, Fahrstuhlkabine	Steigende Elektrizität (3-Schicht-Blitze)
3	Open Sea	Offener See, Leuchtturm	Steigende Flut (Mehrschicht-Meer mit Schaumkronen)

2. Feinkonzept

2.1 Spielfluss & Zustände

Das Spiel verwendet eine klar definierte Zustandsmaschine mit zehn Zuständen (GamePhase):



Weitere Zustände: DEV_BROWSE (freie Kamera für Entwicklung, F2 im PLAYING-Zustand), AUDIO_MENU (Lautstärke-Einstellungen vom Start-Screen aus).

Start-Screen: Spieltitel, Schwierigkeitsauswahl (Explorer / Adventurer / Enlightened), Steuerungshinweise. **LEVEL_INTRO:** Dialogue-Bubble mit Level-spezifischem Text + Countdown vor dem Start. **PLAYING:** Alle Spielmechaniken aktiv; HUD sichtbar. **LEVEL_OUTRO:** Bubble nach Aktivierung des Outro-Triggers; Gefahr blendet aus. **GAMEOVER:** Aktueller Score + Highscore; Neustart ohne Seitenreload.

2.2 Steuerung

Aktion	Tastatur	Maus
Bewegen (links/rechts)	A/D oder Pfeiltasten Links/Rechts	—
Springen	Leertaste	Linksklick
Push (Aktion)	Z	Rechtsklick

Keine Diagonal-Eingabe. Bewegung ist sofort, kein Beschleunigungsmodell.

2.3 Spielerfigur

Die Spielerfigur ist eine **ausgestopfte Katze** (Kuscheltier-Ästhetik), keine realistische Katze. Eigenschaften:

- **Hitbox:** 32 × 32 px (Welt-Koordinaten)
- **Darstellungsgröße:** 96 × 96 px (1,5-fache Skalierung der 64-px-Quell-Sprites)
- **Ausrichtung:** Horizontal gespiegelt bei Richtungswechsel (via `ctx.scale(-1,1)`)
- **Ankerpunkt:** Unterkante der Hitbox bündig mit Plattformoberfläche

2.4 Sprung & Bewegungsmechanik

Sprunglogik:

- Sprung nur möglich wenn `player.onGround === true`
- Sprungausrösung setzt `vy = JUMP_VELOCITY` (negativ, aufwärts)
- `onGround` wird jede Frame durch Kollisionsprüfung gesetzt; kein Coyote-Time-System

Physik:

- Schwerkraft: konstant positiver Y-Wert, addiert auf `vy` pro Frame
- Alle Geschwindigkeiten in px/s, skaliert mit `dt / 1000` (semi-fixes Zeitschritt)
- Delta-Cap bei 50 ms verhindert Physik-Explosion bei Tab-Wechsel

Kollisionsmodell (One-Way AABB): Eine Plattform stoppt die Figur nur wenn alle drei Bedingungen gleichzeitig erfüllt sind: 1. Horizontale Überlappung (Hitboxen überschneiden sich auf X-Achse) 2. Spieler war im Vorframe oberhalb der Plattform 3. Spieler bewegt sich nach unten (`vy > 0`)

Bei Landung: `vy = 0, onGround = true`.

2.5 Aktions-Taste (Z / Rechtsklick)

- Taste Z / Rechtsklick löst eine Pfoten-Aktion aus (250 ms Animation)
- Sprite-Wechsel: `push_rise` (am Boden oder kurz nach Sprung) / `push_peak` (in der Luft)

- Verwendung: Ballon fangen, Wespen abwehren, Level-Abschluss-Objekte aktivieren (Glocke in L2, Hebel in L3)
- Kein Push-System für Items. Die Kletter-Kiste in L2 ist ein statisches Objekt im Outro und nicht interaktiv

2.6 Plattformen

Plattformtypen:

Typ	Verhalten	Visuell	Level
Normal	Stabil, dauerhaft	Levelthema-Sprite (z.B. Jalousie)	Alle
Brüchig (Crumble)	Beginnt zu bröckeln nach Landung, verschwindet nach kurzer Zeit	Rissig gelb → Zerbröckelnd gelb (Zeile 4)	Alle
Sinkend (Cloud-Sink)	Sinkt langsam wenn Katze darauf steht (max. 60 px), steigt danach automatisch zurück. Katze wird mit der Plattform nach unten gezogen.	Wolken-Sprite (L3 Leuchtturm-Setting)	L3

Generierung:

- Plattformen werden prozedural generiert (Endlos-Scroll nach oben)
- Mindestabstand und maximaler Abstand begrenzen Sprungreichweite
- Plattformbreite variiert, zusammengesetzt aus: linker Kappe + gekachelter Mitte + rechter Kappe

Level 1 — Jalousie:

- Sprite-Sheet mit 7 Zeilen: Zeilen 1/2/3/5/6 = intakt (zufällig zugewiesen), Zeile 4 = gerissen, Zeile 7 = zerbröckelnd

2.7 Gefahren-Mechanik (Hazard)

Jedes Level hat eine eigene, stetig steigende Gefahr. Alle drei Gefahren teilen dieselbe Physik (Anstiegsgeschwindigkeit, Beschleunigung, Kollisionserkennung), unterscheiden sich aber visuell:

- **Level 1 (Smog):** Geschichtete Gradientenbänder mit Kosinus-Wölbung am oberen Rand und Glüheffekt
- **Level 2 (Elektrizität):** Drei unabhängige Blitzschichten mit unterschiedlichen Frequenzen, Ankerpunkten und pulsierender Transparenz
- **Level 3 (Flut):** Zwei halbtransparente Wellenschichten als glatte Per-Pixel-Sinuskurven (tiefe langsame Dünung in dunklem Navy, darüber mittlere Welle in Teal-Blau) sowie eine stürmische Oberflächenschicht aus zusammengesetzter Sinuskurve (zwei summierte Frequenzen) mit Tiefengradient und weißen Schaumkronen an Wellenkämmen
- Kontakt mit der Gefahr = Leben verlieren

- Die Gefahr existiert in Weltkoordinaten; die Kamera folgt dem Spieler nach oben
- **Obergrenze:** L1-Smog stoppt 22 px unterhalb des Dachs (visuell bündig). L2/L3 steigen bis auf `levelGoalY` — die Gefahr füllt den gesamten Schacht/Turm
- **Ausblenden:** Beim Auslösen des Zielfeldes blendet die Gefahr über 1,2 s aus

2.8 Leben & Scheitern

- **Start-Leben:** abhängig vom Schwierigkeitsgrad (Explorer: 5, Adventurer: 3, Enlightened: 2) — dargestellt als Cat-Icons im HUD (max. 9 sichtbar). Siehe Abschnitt 2.13.
- **Verlust eines Lebens:** Kontakt mit der Gefahr ODER Herunterfallen unter den Bildschirmrand ODER Kontakt mit dem Wespenstachel
- **Game Over:** Bei 0 Leben
- **Respawn:** Aktuell kein Mid-Level-Respawn (direkter Game Over bei 0 Leben)
- **Zusatzleben (Ballon-Kollektible):** Ein steigender Ballon erscheint in jedem Level. Kontakt mit Z-Taste / Rechtsklick (Pfote muss untere 40 % des Ballons berühren) gibt ein zusätzliches Leben. Der Ballon bewegt sich mit horizontaler Sinusbewegung (50 px Amplitude, 1,8 s Periode) und steigt mit 120 px/s. Wird er nicht gefangen, bevor er `levelGoalY` erreicht, verschwindet er.

2.9 Punkte & Fortschritt

- **Score:** Zurückgelegte Höhe (in Weltkoordinaten) über dem Startpunkt
- Wird kontinuierlich berechnet: `score = max(0, spawnY - player.y)`
- HUD zeigt aktuellen Score in Echtzeit (englisches Label „SCORE“)
- **Highscore:** Lokal im Browser gespeichert (LocalStorage)

2.10 Kamera & Weltkoordinaten

- Alle Spielentitäten existieren in **Weltkoordinaten** (nicht Bildschirmkoordinaten)
- Kamera: einmaliges `ctx.translate(0, -cameraY)` vor dem Render aller Weltobjekte
- HUD wird **nach** `ctx.restore()` gezeichnet (immer in Bildschirmkoordinaten)
- Kamera folgt dem Spieler nach oben; Scroll nach unten nur bis zum Ausgangspunkt

2.11 Gegner & Gefahren pro Level

Gefahren (Hazard-System):

Level	Gefahr	Beschreibung
1 (Stadt)	Steigender Smog	Geschichtete Gradientenbänder mit Kosinus-Wölbung; steigt bis 22 px unterhalb des Zieldachs
2 (Aufzugschacht)	Steigende Elektrizität	3-Schicht-Blitze mit unabhängigen Frequenzen und pulsierender Transparenz; steigt bis <code>levelGoalY</code>
3 (Offener See)	Steigende Flut	Zwei halbtransparente Sinuswellen-Schichten (Dünung + Wasserkörper) plus stürmische Oberfläche mit

		Compound-Sinus, Tiefengradient und Schaumkronen an Wellenkämmen; steigt bis `levelGoalY`
--	--	--

Wespen-Gegner (implementiert, `src/enemies.js`):

Wespen patrouillieren horizontal auf festen Y-Positionen (Welt-Koordinaten). Sie folgen einem einfachen Zustands-Automaten: Patrol → Reverse → Patrol. Zusätzlich schwingen sie vertikal mit kleiner Sinusamplitude (20 px, 1,5 Hz).

Anzahl Wespen pro Level wird vom Schwierigkeitsgrad multipliziert (siehe 2.13). Basiswerte (Adventurer-Default):

Level	Basisanzahl	Verhalten
1 (Stadt)	8	Patrol 4–8 s, dann Richtungswechsel
2 (Aufzugschacht)	12	Wie L1, höhere Dichte im engen Schacht
3 (Offener See)	16	Höchste Anzahl, maximale Schwierigkeit

Wespen-Interaktionen:

Interaktion	Auslöser	Ergebnis
Stich	Wespenstachel (Hitbox am Hinterleib) trifft Katze	Katze: –1 Leben + 1,2 s Unverwundbarkeitszeit; Wespe: kurze Knockback-Bewegung, überlebt
Stomp	Katze landet auf dem oberen Bereich der Wespe (Stomp-Hitbox: 12 px)	Wespe: Schrumpf-/Ausblend-Animation über 0,5 s, dann entfernt; Katze: kleiner Abprallsprung

2.12 Levelstruktur im Detail

Level 1 — City (Stadtsetting):

- Plattformthema: Jalousien an Stadtgebäuden
- Unsichtbare Plattformen auf Dekor-Elementen (Mülltonnen, Türrahmen, Gesims)
- Einstiegslevel, niedrigste Schwierigkeit
- Führt Sprung- und Bewegungsmechanik ein; Gefahr: steigender Smog
- Outro-Trigger: Windrad auf dem Zieldach (JS-gezeichnet, vier rotierende Schaufeln); Aktivierung mit Z am Reach-Punkt schließt das Level ab

Level 2 — Elevator Shaft (Aufzugschacht):

- Start: Spieler steht im Fahrstuhl (Erdgeschoss). Die Katze klettert durch eine Decken-Luke in den Schacht.

- Decken-Mechanik: Nur die Luke (x=164–311) ist durchdringbar — der Rest der Fahrstuhldecke hat unsichtbare Kollider. Gleiches Prinzip am oberen Schachtausgang (x=138–337).
- Schachtboden: Linke und rechte Randbereiche begehbar. Die mittlere Öffnung (x=172–300) hat keinen Boden — Katze fällt zurück in den Fahrstuhl und verliert ein Leben.
- 404-Display: Der Fahrstuhl zeigt ein „404“-Display über den Türen; die Oberkante dient als unsichtbare Plattform (y=301, x=210–262).
- Hintergrund: Fahrstuhl-Sprite (Erdgeschoss) → Schacht-Boden-Sprite (überlappt Fahrstuhldecke nahtlos) → getilede Schacht-Wände (alternierend) → Schacht-Top mit goldenem Balken
- Parallax-Mid-Layer: Kabel/Rohr-Sprites (0,9×-Faktor) hinter den Schachtwänden
- Gefahr: Steigende Elektrizität als 3-Schicht-Blitzsystem; erreicht `levelGoalY`
- Kletter-Kiste: Statisches Dekorations-Objekt im Outro-Bereich. Push-Mechanik wurde am 25.04.2026 vollständig gestrichen. Kein interaktives Verhalten mehr.
- Outro-Trigger: Glocke auf einem Stand am Schacht-Ausgang (`bell_spritesheet.png` + `bell_stand.png`, selbst gepixelt); Aktivierung mit Z schließt das Level ab

Level 3 — Open Sea (Offener See / Leuchtturm):

- Setting: Leuchtturm an einer felsigen Küste am offenen Meer (ersetzt ursprüngliches Raketen-Setting; Entscheidung 30.03.2026)
- Start: Spieler steht auf dem Meeresgrund / Startplattform. Ziel: Spitze des Leuchtturms.
- Plattformthema: Felsvorsprünge, Wellenbrecherstufen, Balkone am Turm
- Hintergrund: Weite See, Horizont, Sonne mit Puls-Animation; Leuchtturm als zentrale vertikale Struktur (Stein/Backstein)
- Shared-Layers (Himmel, Wolken) bleiben unverändert
- Gefahr: Steigende Flut als Mehrschicht-Meer (zwei halbtransparente Sinuswellen plus stürmische Oberfläche mit Compound-Sinus und Schaumkronen); steigt bis `levelGoalY` und passt thematisch zum Leuchtturm-Setting
- Outro-Trigger: Hebel auf dem Leuchtturm-Balkon (`lever_mid.png`, Skalierung 0,68); Aktivierung mit Z schließt das Level ab

2.13 Schwierigkeitsgrade

Drei Modi mit globalen Multiplikatoren auf Hazard-Speed, Wespen-Anzahl und Cloud-Drift in Level 3. Auswahl auf dem Start-Screen, Default ist Adventurer.

Modus	Hazard-Speed ×	Wespen ×	Cloud-Drift ×	Start-Leben
Explorer	0,80	0,60	0,00 (still)	5
Adventurer	1,00	1,00	0,80	3
Enlightened	1,25	1,30	1,40	2

Multiplikatoren werden einmal beim Run-Start gelesen und gelten für die ganze Sitzung. Der gewählte Modus wirkt direkt auf `src/hazards.js` (Speed), `src/enemies.js` (Wespen-Basiswerte 8/12/16) und L3-Wolken-Drift in `src/platforms.js`. Lives-Cap im HUD: 9 (sichtbare Icons).

3. Designkonzept

3.1 Visueller Stil & Stimmung

Kernstimmung: „Cozy Danger“

Das Spiel kombiniert warme, plüschige Pixel-Ästhetik mit einer wachsenden Bedrohung. Die Figur und die Welt fühlen sich weich und einladend an. Die aufsteigende Flut erzeugt Spannung, ohne die Atmosphäre in Richtung Horror zu kippen.

Stilmerkmale:

- Pixel Art, 64px-Basis für Figur-Sprites
- Keine schwarzen Outlines — Kontrast durch Farbabstufungen
- Satte, gesättigte Farben mit kontrollierter Palette (16 Farben, fest definiert)
- Weiche Formen für Figur und Plattformen; harte Kanten nur für Architektur

Verbotene Stilmittel:

- Kein Antialiasing (imageSmoothingEnabled = false durchgehend)
- Keine CSS-Effekte auf dem Canvas
- Keine externen Fonts mit Blur/Shadow

3.2 Farbpalette

Die Palette ist auf 16 Farben festgelegt und unveränderlich. Kein Schwarz (#000000) als Outline.

Verbindliche Quelle: docs/STYLE_GUIDE.md — die dort definierte Palette ist die einzig gültige Version. Sie wurde in Phase 04.1 verabschiedet und ist für alle weiteren Phasen gesperrt.

Die folgende Tabelle ist eine Übersicht der Slot-Rollen. Genaue Hex-Werte immer aus dem Style Guide entnehmen.

Slot	Rolle
BG-1	Taghimmel
BG-2	Nachthimmel / tiefer Hintergrund
BG-3	Wolken-Highlight / warmer Akzent
BG-4	Wolken-Schatten / mittlerer Hintergrundton
PLAT-1	Plattformgrundfarbe
PLAT-2	Plattformkante / Schatten
PLAT-3	Brüchige Plattform (gerissener Zustand)
PLAT-4	Brüchige Plattform (zerbröckelnder Zustand)
CAT-1	Katzenkörper (Plüschbeige)
CAT-2	Katzenschatten
CAT-3	Katzenbauch / Highlight
CAT-4	Katzenohr-Akzent
CAT-5	Katzenknopfaugen
WATER-1	Flut-Hauptfarbe
WATER-2	Wellenkamm-Highlight

UI-1	HUD-Text / UI-Elemente
------	------------------------

3.3 Sprite-Design: Spielerfigur

Die ausgestopfte Katze ist als Plüschtier konzipiert — erkennbar durch Nähte, Knopfaugen und weiche Körperproportionen.

Anatomie:

- Körper: CAT-1 (Beige), mit CAT-2 für Highlights, CAT-3 für Schatten
- Augen: Runde schwarze Knöpfe (CAT-4), kein Glanz
- Nähte: dünne CAT-3-Linien
- Proportionen: Kopf ca. 40% der Gesamtgröße (cartoonhaft übergroß)
- Kein Mund (Plüschtier-Stil)

Sprite-Set (Implementiert):

Alle 7 Posen sind in einem einzigen Spritesheet zusammengefasst:

Visuals/cat/animation_sheet.png

Index	Pose	Zustand
0	Idle	Stehend auf Plattform
1	Rise	Aufstieg nach Sprung
2	Walk 1	Laufzyklus Frame 1
3	Walk 2	Laufzyklus Frame 2
4	Push Rise	Aktions-Pfote am Boden / tief in der Luft
5	Push Peak	Aktions-Pfote auf Höhepunkt
6	Peak	Höhepunkt / freier Fall

Sprite-Auswahl-Logik (Priorität von oben nach unten): 1. $pushTimer > 0 \rightarrow push_rise$ (am Boden oder kurz nach Sprung) / $push_peak$ (hoch oben in der Luft) 2. $onGround + vx \neq 0 \rightarrow walk_1/walk_2$ im Wechsel (alle 150 ms) 3. $onGround + vx = 0 \rightarrow idle$ 4. Kurz nach Sprung ($bounceTimer > 0,20$) $\rightarrow idle$ (erste 40 ms) 5. Aufsteigend ($bounceTimer > 0,05$) $\rightarrow rise$ 6. Schnell fallend ($vy > 600$) $\rightarrow rise$ (Vorahnungs-Frame) 7. Sonst $\rightarrow peak$

Y-Offsets (Pixelkorrektur für transparente Randbereiche, 1,5× skaliert):

Index	Pose	Offset
0	idle	16 px
1	rise	16 px
2	walk_1	16 px
3	walk_2	16 px
4	push_rise	16 px
5	push_peak	12 px
6	peak	12 px

3.4 Hintergrund & Parallax

Das Hintergrundsystem besteht aus 5 Ebenen mit unterschiedlichen Scrollgeschwindigkeiten (Parallax):

Ebene	Asset	Parallax-Faktor	Beschreibung
1	sky_day.png	0,30×	Taghimmel (BG-1)
2	sky_night.png	0,30×	Nachthimmel (BG-2), crossfade mit Höhe
3	stars.png	0,10×	Sterne, erscheinen ab $t=0,3$
4	clouds_bright.png	0,60×	Helle Wolken, horizontaler Drift 15 px/s
5	clouds_dark.png	0,60×	Dunkle Wolken

Tag-zu-Nacht-Übergang: Der Übergang von Tag- zu Nachtatmosphäre erfolgt graduell anhand der Spielhöhe. Variable t (0 = Boden, 1 = Levelziel). Sterne werden sanft eingeblendet ab $t=0,3$, voll sichtbar ab $t=0,7$.

Wolkenbewegung: Horizontaler Drift bei 15 px/s (bright) und 4,5 px/s (stars) — kreiert ein lebendiges, atmendes Bild auch ohne Spielereingabe.

3.4.1 Notfall-Fix: Steinboden-Overlay (Level 3 Leuchtturm)

Status: aktiv, bleibt für die Schulabgabe bestehen.

Situation: Die Leuchtturm-Basis in Level 3 wird über ein Sprite-Sheet gerendert (Visuals/backgrounds/level_3_sea/lighthouse_sheet2.png). Die Ursprungs-Leinwand in Pixelorama war 480 × 640 px, der Steinboden ging dort bis an den linken und rechten Rand.

Problem: Nach dem Export aus Pixelorama wurde das Sprite in Illustrator weiterverarbeitet (Abstände justiert, alle neun Kacheln zu einer Sheet-Datei zusammengebaut). Beim Sheet-Export schneidet Illustrator jede Einzelzelle auf ihre sichtbaren Pixel zu. Die Basiszelle erscheint im fertigen Sheet daher nur noch 439 px breit, statt wie gewünscht 480 px. Im Spiel hatte der Steinboden dadurch links und rechts je ca. 20 px transparenten Rand und wirkte zu schmal.

Saubere Lösung (nicht umgesetzt, da Zeitaufwand): Den Sheet-Export-Schritt in Illustrator so anpassen, dass jede Zelle in ihrer vollen 480-px-Leinwandbreite exportiert wird (keine Content-Trim-Option). Danach die Sprite-Koordinaten im Code aktualisieren.

Notfall-Fix (umgesetzt): Ein separates Overlay-PNG

(Visuals/backgrounds/level_3_sea/lh_00_quick-fix.png) wird in `src/background.js` über die Basiszelle gezeichnet und deckt den transparenten Randbereich ab. Das Overlay nutzt seine native Pixelgröße (kein Stretching), ist auf $x = 240$ zentriert und folgt der Kamera über denselben `camShift + GroundOffset`, der auch die Basiszelle positioniert. Die Unterkante des Overlays ist über die Konstante `_LH_STONE_FIX_ANCHOR_Y = 633` an die Steinlinie der Basiszelle angekoppelt.

Code-Stellen:

- **Loader:** `src/background.js`, umrahmt mit einem `EMERGENCY FIX — LOCKED — DO NOT CHANGE` Kommentarblock.
- **Draw-Call:** `_drawL2Lighthouse(ctx, camShift)` in `src/background.js`, direkt nach dem Zeichnen der Basiszelle.

Begründung der Entscheidung: Der Pipeline-Fix hätte mehrere Export-/Re-Import-Iterationen gebraucht (Illustrator-Export-Einstellungen prüfen, Sheet neu aufbauen, Sprite-Koordinaten im Code neu vermessen, alle neun Zellen gegenprüfen). Angesichts der Abgabeplanung ist ein Overlay-PNG der pragmatische Weg: es verändert die Pipeline nicht, beeinflusst keine anderen Zellen des Sheets, und das Ergebnis ist visuell identisch mit dem Zielzustand.

Entscheidung: Der Overlay-Fix bleibt in der Abgabeverision. Die Pipeline-Ursache ist dokumentiert, damit nachvollziehbar ist, warum der Fix existiert und welcher saubere Weg in einer Post-Abgabe-Iteration möglich wäre.

3.5 Plattform-Design

Level 1 — Jalousien (Stadtsetting):

- **Sprite-Sheet:** `Visuals/platforms/level1_city/jalousie_sheet.png`
- 7 Reihen im Sheet: 5 intakte Zustände, 1 gerissen (gelb, Reihe 4), 1 zerbröckelnd (gelb, Reihe 4)
- 3-teiliges Rendering: linke Kappe + gekachelte Mitte + rechte Kappe
- Zufällige Zeile pro Plattform bei Generierung (aus Zeilen 1,2,3,5,6)

Level 3 — Wolken (Leuchtturm/Offener See):

- **Plattformtyp:** Cloud-Sink (sinkende Wolken)
- 3-Type-Mix bei Generierung: normal, crumble, cloud-sink
- Cloud-Sink-Plattform sinkt mit dem Gewicht der Katze (max. 60 px), steigt dann eigenständig zurück
- Katze wird aktiv nach unten mitgezogen — erzeugt Zeitdruck in Kombination mit der steigenden Flut

3.6 UI & HUD

Sprache: Alle UI-Texte in Englisch (Migration 2026-04-21 von Spanisch).

Start-Screen:

- Titel: „Soggy Moggy“ (groß, zentriert)
- Untertitel / Spielanleitung: kurze Steuerungsübersicht
- Schwierigkeits-Auswahl: Explorer / Adventurer / Enlightened (Default Adventurer)
- Startaufforderung: „Press SPACE to play“

HUD (während des Spiels):

- Schwarzes Banner (`rgba(0,0,0,0.50)`), Höhe 68 px) hinter den HUD-Elementen
- Oben links: SCORE (x=12, y=24) und LEVEL (x=12, y=46), beide in Caps
- Oben rechts: Lives als Cat-Icons (32×26 px), rechts-ausgerichtet, max. 9 Icons sichtbar
- Farbe: UI-1 auf schwarzem Banner

Game-Over-Screen:

- Titel: „Game Over“ (stilisiert)
- Aktueller Score + Highscore
- Neustart-Aufforderung: „Press SPACE to restart“

Design-Regeln für UI:

- Pixel-font oder system-font (Arial) in Pixel-Größen
- Keine halbtransparenten Overlays ohne feste Farbbegrenzung
- Alle UI-Elemente außerhalb der Weltkoordinaten (nach `ctx.restore()`)

3.7 Animationssystem

Das Animationssystem ist frame-basiert ohne externe Bibliothek:

- **Laufzyklus:** 2 Frames (`walk_1`, `walk_2`), Wechsel alle 150 ms via `performance.now()`
- **Sprite-Wechsel:** Zustandsbasiert, kein Tweening
- **Bounce-Timer:** 240 ms nach Sprung aktiv — verhindert sofortigen Sprite-Wechsel
- **Push-Timer:** Aktiv solange Wurf-Aktion läuft

Kein Sprite-Blending oder Fade zwischen Frames — harte Cuts entsprechen dem Pixel-Art-Stil.

3.8 Audio-System

Das Spiel verwendet **HTMLAudio API** für alle Sound-Events und Musik (implementiert in `src/audio.js`). Die Web Audio API wurde bewusst nicht eingesetzt, da Firefox XHR-Anfragen auf `file://` blockiert (Begründung vollständig in Abschnitt „Bekannte Einschränkungen“).

SOUNDS-Map (31 Einträge):

Kategorie	Keys	Beschreibung
SFX — Spieler	<code>jump</code> , <code>land</code> , <code>damage</code> , <code>balloon_collect</code> , <code>stomp_bounce</code> , <code>respawn</code>	Bewegungs- und Interaktions-Sounds
SFX — UI	<code>game_over</code> , <code>level_complete</code> , <code>menu_click</code> , <code>menu_nav</code> , <code>countdown_tick</code>	Interface-Feedback und Menü-Navigation
SFX — Outro-Trigger	<code>windrad</code> , <code>bell</code> , <code>l3_lever</code> , <code>water_drain</code> , <code>l1/l2/l3_outro_bubble</code>	Levelabschluss-Sounds, jeweils levelspezifisch
SFX — Gegner	<code>wasp_sting</code> , <code>wasp_death</code> , <code>wasp_buzz</code>	Wespen-Interaktionen
SFX — Gefahren (Ambient)	<code>smog_ambient</code> , <code>electricity_ambient</code> , <code>flood_ambient</code>	Proximity-gesteuerte Umgebungsgeräusche
SFX — Plattformen	<code>crumble</code> , <code>electro_crumble</code>	Brüchige Plattform-Events (L1 / L2)
Musik	<code>music_start</code> , <code>music_l1</code> , <code>music_l2</code> , <code>music_l3</code> , <code>music_victory</code>	Start-Menü + ein Track pro Level + Sieg-Jingle

Wichtige Systeme:

Preload-Cache: Alle SFX-Einträge werden beim Seitenlade-Vorgang in HTMLAudio-Elemente vorgeladen. Musik wird bei Bedarf gestreamt.

Phasen-Hooks: Übergänge zwischen GamePhase-Zuständen triggern automatisch passende Sounds. Beispiele: GAMEOVER → `game_over.mp3` + Game Over Screen mounten; LEVEL_OUTRO (L1) → `l1_outro_bubble.mp3` + Musik ausblenden; LEVEL_COMPLETE (L3) → Highscore speichern + Success Screen mounten.

Wasp-Buzz-Proximity: Ein looping HTMLAudio-Element (`wasp_buzz.mp3`) dessen Lautstärke quadratisch mit dem Abstand zur nächsten lebenden Wespe skaliert (Reichweite 350 px, Maximum 55 % des SFX-Volumes).

Hazard-Ambient-Proximity: Die drei levelspezifischen Umgebungsgeräusche (Smog, Strom, Flut) werden über denselben Proximity-Mechanismus gesteuert und verstärken sich mit sinkendem Abstand zum Hazard.

Trim-Werte: Pixabay-MP3-Dateien enthalten typischerweise 20–50 ms encoder-bedingte Stille am Dateianfang (LAME/FFmpeg Priming-Frames). Diese wird über `start`-Offsets im SOUNDS-Mapping übersprungen, um wahrnehmbare Verzögerungen zu minimieren.

Audio-Grundsatz: Keine aggressiven oder erschreckenden Sounds. Alle Klänge unterstützen die „cozy danger“-Stimmung des Spiels.

3.9 Typografie

Verwendung	Schriftart	Größe	Farbe
HUD-Score	Pixel-Font / Arial	16–18 px	UI-1
Spieltitel (Start)	Pixel-Font / Arial Bold	32–40 px	UI-2
Hinweistexte	Arial	14 px	UI-1

Keine mehr als zwei Schriftgrößen pro Screen. Alle Texte linksbündig oder zentriert — kein Flattersatz rechts.

3.10 Produktionswerkzeuge (Pixel Art)

Alle Sprites wurden manuell mit **Pixelorama** gezeichnet, einem Open-Source-Pixelart-Editor. Quelldateien haben die Endung `.pxo`. Weitere Informationen und Download:

<https://pixelorama.org/>

Einzelne Sprites wurden mit **Adobe Photoshop** zusammengesetzt, zum Beispiel beim Erstellen von Spritesheets aus Einzelteilen. Diese Dateien sind an der Endung `.psd` erkennbar.

Software	Verwendung	Erkennbar an
Pixelorama	Erstellung aller Pixel-Art-Sprites (manuell gezeichnet)	<code>.pxo`</code> Quelldatei
Adobe Photoshop	Zusammensetzen von Sprites / Spritesheets	<code>.psd`</code> Datei

Welche Datei mit welchem Tool erstellt wurde, wird im **Medienkatalog** am Projektende vollständig dokumentiert.

Bekannte Einschränkungen (Stand Abgabe)

Audio-Latenz (HTMLAudio auf file://)

Das Spiel läuft ohne Webserver direkt vom USB-Stick via `file://`. Aus diesem Grund wird HTMLAudio statt der Web Audio API verwendet. HTMLAudio hat auf Windows eine systeminterne Latenz von ca. 50 bis 90 ms pro Sound-Event. Zusätzlich enthalten Pixabay-MP3-Dateien eine encoder-bedingte Stille von ca. 20 bis 50 ms am Dateianfang (LAME/FFmpeg Priming-Frames).

Konkrete Auswirkungen:

- Sprung-Sound (`jump.mp3`): spielt mit ca. 70 bis 140 ms Verzögerung nach dem tatsächlichen Sprung. Bei schnellen Mehrfachsprüngen kann der Sound abgeschnitten oder ausgelassen wirken. Die Datei selbst ist länger als nötig; ein sauberer Schnitt auf den kurzen Kernton war technisch nicht stabil umsetzbar.
- Landungs-Sound (`land.mp3`): Einsatz ab `start: 0.02 s` kalibriert, minimale wahrnehmbare Verzögerung verbleibt.
- Alle weiteren SFX: selbe Latenzcharakteristik, bei einmaligen Events (Game Over, Level Complete) nicht wahrnehmbar störend.

Ursache und Entscheidung:

Die Web Audio API würde Sample-genaue Wiedergabe ermöglichen (ca. 20 bis 35 ms Latenz). Sie benötigt jedoch `fetch()` oder XHR zum Laden von Audio-Puffern. Firefox blockiert XHR auf `file://` mit einem CORS-Fehler. Da der Browser des Prüfers unbekannt ist und das Spiel auf jedem Browser laufen soll, wurde die Web Audio API verworfen. HTMLAudio lädt über den HTML-Parser, nicht über XHR, und funktioniert in allen Browsern zuverlässig auf `file://`.

Die Latenz ist eine dokumentierte technische Einschränkung des Abgabe-Formats, kein Implementierungsfehler.

Anhang: Technische Architektur-Übersicht

Dateistruktur (Quellcode):

```
src/
├─ main.js                – Hauptschleife, Initialisierung, Render-Orchestrierung,
Outro-Trigger-System
├─ game-state.js          – Zustandsmaschine (GamePhase, GameState, DIFFICULTY-
Tabelle)
├─ input.js               – Tastatur-/Mauslistener
├─ player.js              – Spielerfigur, Sprung, Sprite-Logik, Pfoten-Aktion
├─ platforms.js           – Plattformgenerierung, Kollision, Sprite-Rendering,
unsichtbare Kollider (L2/L3)
├─ hazards.js             – Hazard-System (Smog, Elektrizität, Flut) – eines pro Level
├─ enemies.js             – Wespen-System (Patrol, Stomp, Knockback, Spawn,
getNearestWaspDist)
├─ background.js          – Parallax-Hintergrundsystem, Crossfade, Level-spezifische
Renderer
```

— audio.js	— HTMLAudio SFX + Musik; SOUNDS-Map, Preload,
playSound/playMusic/fade,	Proximity
— dialogue.js	— Dialogue-Bubble-System (LEVEL_INTRO/OUTRO pro Level,
Override-Tabellen)	
— start-screen.js	— Start-Screen React-Overlay (FlatLayoutB,
Schwierigkeitsauswahl)	
— gameover-screen.js	— Game Over React-Overlay (Score, Highscore, Neustart)
— success-screen.js	— Success/Congrats React-Overlay mit Konfetti-Animation (L3
Ziel)	
— levelcomplete-screen.js	— Level-Complete React-Overlay (L1/L2 Ziel, Score-Anzeige)

Update-Reihenfolge pro Frame (load-bearing): 1. updatePlayer(dt) 2.

updatePlatforms(dt) 3. checkPlatformCollisions() 4. updateEnemies(dt) — muss nach Kollisionsprüfung laufen (liest player.prevY) 5. updateCamera() 6. Render (Welt dann HUD)

Zeitschritt:

- Semi-fixes System: $dt = \min(\text{elapsed}, 50)$ in ms
- Alle Physik-Werte in px/s, multipliziert mit $dt / 1000$

Anhang: KI-Assistenz und Prompting-Strategie

Dieses Projekt wurde mit Claude Code (Anthropic) als strukturiertem Pair-Programming-Werkzeug entwickelt. KI-generierter Code entstand immer nach einem verifizierten Plan; was gebaut, geändert oder verworfen wurde, war durchgehend die Entscheidung des Autors. Jede Phase folgte dem Ablauf Recherche → Annahmen prüfen → Plan ausführen → Ergebnis verifizieren, nie direkt "schreib mir Code für X".

Entscheidende Eingriffe durch den Autor: MVP-Grenze klar gesetzt und gehalten (Level 4 gestrichen, Push-Mechanismus verworfen), Dozenten-Feedback direkt ins Planungssystem eingearbeitet, technische Browseranforderungen (file://, kein Build-Tool, Firefox-kompatibel) als feste Rahmenbedingung von Anfang an definiert. Zwei konkrete KI-Korrekturen nach eigenem Testen: Web Audio API (Chromium-only, nicht geeignet) wurde durch HTMLAudio ersetzt; Babel/JSX (XHR-Abhängigkeit auf file://) wurde durch React.createElement ohne Transpiler ersetzt.

Vollständige Dokumentation inkl. GSD-Kommandos, konkreter Beispiele und Phasenprotokoll: [\[docs/prompting-strategy/prompting-strategy-final-2026.md\]](#)(docs/prompting-strategy/prompting-strategy-final-2026.md)

Dokument erstellt: März 2026 Letzte Aktualisierung: 27.04.2026